TDSnap 학습 주석본

한 줄 요약

Intel TDX 기밀 VM 안에서 FaaS 함수를 돌리면 시작이 3~4초로 느려진다. TDSnap은 검증을 마친 VM 상태를 암호화된 스냅샷으로 저장해 두고 요청이 오면 거기서 복원해서, 보안을 유지한 채 시작 시간을 최대 40배 줄인다. 출처: Kim, Shin, You, DAC 2026, CC BY 4.0. 유명성 교수님 교신저자.

읽는 순서

1) 초록과 그림 1 (무엇을 왜 하는가) 2) 2.1절 TDX 설명 (용어가 몰려 있다) 3) 2.3절 두 가지 도전 과제 4) 3장 설계는 그림 3·4와 함께 5) 4장 평가는 그림 5와 표 1만 먼저. 처음 읽을 때는 노란색만 따라가고, 두 번째에 초록 용어 메모를 읽는 것을 권한다.

표시 규칙

노랑: 핵심 주장, 결과 수치

초록: 전문용어. 오른쪽에 풀이

분홍: 한계, 가정, 위협 모델

①②… 빨간 번호 = 오른쪽 여백의 같은 번호 설명. → 표시 = EN683·한화비전과 연결되는 지점.

메모는 원문과 일반 지식에 근거한다. 원문에 없는 해석은 (추정)으로 표시했다. 원문: CC BY 4.0. 형광펜·메모는 원문에 추가한 것이다.

용어집 (등장 순서)

FaaS: 요청이 올 때만 함수를 띄워 실행하고 끝나면 지우는 클라우드 방식. 예: AWS Lambda. 서버를 계속 켜 두지 않으니 싸다.

CVM(기밀 VM): CPU가 VM 메모리를 암호화해서 호스트 OS와 하이퍼바이저도 내용을 못 보게 하는 VM. Intel TDX, AMD SEV-SNP가 대표적.

cold start: 함수를 처음부터 띄우는 과정. 이 시간이 1초를 넘으면 FaaS로 쓰기 어렵다.

multi-tenant: 서로 다른 고객(tenant)의 코드가 같은 서버에서 함께 도는 구조. 옆 고객이 공격자일 수 있다.

TD: TDX가 만드는 격리 VM 하나. 메모리와 vCPU 레지스터가 하드웨어로 암호화·보호된다. → 리눅스가 못 들여다보는 보안 영역이라는 점에서 EN683의 EIP-130 안쪽과 비슷하다(비유).

원격 증명: 장치가 자기 소프트웨어 측정값(해시)을 하드웨어 키로 서명해 외부 검증자에게 보내는 절차. → 카메라 원격 증명도 같은 개념이다.

TD 템플릿: TD를 만들 때 매번 같은 부분(바뀌지 않는 구조)을 미리 만들어 둔 틀. 매번 새로 만들지 않아 빠르다.

working set(작업 집합): 요청을 처리할 때 실제로 자주 쓰는 메모리 페이지들. 이것만 먼저 올리고 나머지는 필요할 때 올린다.

microVM: 장치 에뮬레이션을 최소로 줄인 아주 가벼운 VM. 예: AWS Firecracker. 수백 ms 안에 뜬다.

SEAM: TDX 모듈이 도는 CPU의 특수 실행 모드. 호스트 커널보다 높은 권한으로 TD를 관리한다.

SEAMCALL: 호스트(VMM)가 TDX 모듈에 "TD 만들어 줘, 페이지 넣어 줘"라고 요청하는 CPU 명령. 시스템콜의 TDX판이라고 보면 된다.

SEPT: TD의 게스트 주소를 실제 물리 주소로 바꾸는 페이지 테이블. 호스트가 직접 못 고치고 SEAMCALL을 거쳐야 하며, TD가 승인해야 바뀐다.

CoW: 여러 주체가 같은 페이지를 공유하다가 누군가 쓰려 할 때만 복사본을 만드는 기법. 메모리와 시간을 아낀다.

page fault: 아직 메모리에 없는 페이지에 접근할 때 나는 예외. 보통 이걸 추적해서 어떤 페이지를 쓰는지 안다. TD 안의 fault는 호스트에 안 보인다.

VM exit: VM 실행이 멈추고 제어가 호스트(VMM)로 넘어오는 사건. 호스트가 무언가를 관찰할 수 있는 유일한 순간이다.

tail latency: 지연 분포의 꼬리(P99 등). 평균이 좋아도 가끔 아주 느리면 서비스 품질이 나쁘다.

service TD: 인텔이 제공하는 특수 TD. 마이그레이션 같은 플랫폼 기능을 대신 수행한다. TDSnap 에이전트가 이것으로 만들어진다.

offline / online phase: 미리 스냅샷을 만드는 단계(offline)와 요청이 왔을 때 복원하는 단계(online). 비싼 일은 offline으로 미룬다.

MSK(마이그레이션 세션 키): TD를 옮길 때 페이지를 암호화하려고 TDX 모듈과 에이전트가 만드는 대칭키. TDSnap은 이 키로 스냅샷 페이지를 암호화하고 MAC을 붙인다.

EXPORT SEAMCALL: TD의 private 페이지를 암호화된 형태로 TD 밖으로 내보내는 마이그레이션 명령. 짝은 IMPORT(다시 넣기).

MAC: 키를 가진 쪽만 만들 수 있는 짧은 검증값. 내용이 한 비트라도 바뀌면 검증이 실패해서 변조를 잡는다.

(S)EPT violation: 페이지 테이블에 유효한 항목이 없을 때 나는 예외. 일반 page fault와 달리 반드시 VM exit를 일으켜 호스트가 주소(GPA)를 볼 수 있다. 내용은 여전히 못 본다.

GPA(Guest Physical Address): 게스트가 생각하는 물리 주소. 호스트는 어느 페이지를 건드렸는지(주소)만 알고 데이터는 모른다.

sealing key(봉인 키): CPU 안에서 특정 소프트웨어 신원에 묶여 생성되는 키. 같은 에이전트만 다시 얻을 수 있다. → EN683의 HUK(하드웨어 고유키)로 키를 감싸 저장하는 방식과 같은 발상(추정).

monotonic counter: 증가만 하고 되돌릴 수 없는 하드웨어 카운터. 스냅샷에 이 값을 신선도 태그로 묶어서 옛 스냅샷 재사용(롤백)을 막는다. → 펌웨어 anti-rollback 버전 카운터(OTP)와 같은 역할.

MRTD: TD 생성 시 올린 초기 내용의 측정값. RTMR: 실행 중 추가 측정을 누적하는 레지스터(TPM의 PCR과 비슷). 둘이 TD의 신원이 된다.

on-demand paging: working set에 없는 페이지는 접근할 때 그때그때 IMPORT로 가져온다.

Knative: 쿠버네티스 위에서 FaaS를 돌리는 오픈소스 플랫폼.

QEMU: 리눅스에서 VM을 실행하는 가상화 소프트웨어(VMM 역할). TDX 마이그레이션을 지원하도록 확장했다.

Migration TD(MigTD): 인텔이 공개한 Rust 기반 service TD. TDSnap 에이전트가 이걸 바탕으로 만들어졌다.

Gramine-TDX: TD 안에서 앱 하나를 돌리는 가벼운 라이브러리 OS. 스냅샷 없는 하한 기준선.

REAP, FaaSnap: TDX를 쓰지 않는 최신 스냅샷 시스템. 보안이 없는 대신 빠른 상한 기준선.

CoFunc: TD를 공유하는 기존 연구(USENIX Security 2025). TDX 구현이 공개되지 않아 논문 수치로 비교했다.

SEVeriFast: AMD SEV microVM 부팅 경로를 줄인 연구. 부팅만 빠르게 하고 스냅샷은 다루지 않는다.

SGX: 인텔의 이전 TEE. 프로세스 일부(enclave)만 보호한다. TDX는 VM 전체를 보호한다는 점이 다르다.

Cryonics(SoCC 2023): 같은 저자들(김성중, 유명성 교수님, 신승원 교수님)의 SGX enclave 스냅샷 연구. TDSnap은 이를 VM 단위 TDX로 옮긴 후속작으로 읽힌다(추정).

TDX Connect: TD와 장치(GPU, NIC 등) 사이를 신뢰 채널로 잇는 인텔의 차기 기능. 함수 체이닝 통신에 쓰는 것이 향후 과제.

TDSnap: Enabling Secure Function-as-a-Service with Trusted Domain Snapshots

Seong-Joong Kim
National Security Research Institute
sungjungk@gmail.com

Seungwon Shin
KAIST
claude@kaist.ac.kr

Myoungsung You
University of Seoul
famous@uos.ac.kr

Abstract

① Function-as-a-Service (FaaS) has become a de facto paradigm for deploying cloud applications, yet it exposes sensitive data to untrusted infrastructures. Running functions on ② confidential virtual machines (CVMs), such as Intel TDX, provides strong isolation for data but significantly increases ③ cold-start latency due to CVM initialization. This paper presents *TDSnap*, the first TDX-aware function snapshot system that records a verified function state with its TD and restores it on demand. Using snapshot-bounded attestation and TD-aware working-set identification, *TDSnap* preserves TDX isolation while reducing cold-start latency. Evaluation shows that *TDSnap* reduces cold-start latency by up to ④ 40x and matches non-TDX snapshot systems.

1 Introduction

Serverless computing has become a widely adopted model for building and deploying large-scale cloud applications. Among its various abstractions, Function-as-a-Service (FaaS) [27] is the most prevalent, enabling developers to structure complex applications as collections of lightweight *functions* that execute only in response to predefined events (e.g., client requests). Thanks to its fine-grained resource efficiency and a pay-as-you-go cost model, major cloud providers have widely adopted FaaS as a core service model [5, 11, 21].

As FaaS increasingly handles privacy-sensitive workloads, such as face recognition [9, 10, 20] and financial transaction processing [6, 13], the need for strong security isolation for function data has become paramount. Ensuring data confidentiality and integrity in FaaS platforms is particularly challenging under their multi-⑤ tenant execution model, where functions from different tenants share the same underlying infrastructure. A malicious function can leak sensitive information from co-located functions, while a ⑥ compromised host OS may tamper with data from all tenant functions by abusing its privileged control [16, 18, 23, 24, 26]. These threats underscore the necessity of robust isolation mechanisms to protect per-function data.

Recent advances in confidential virtual machines (CVMs) [22] offer a promising foundation for addressing these challenges. For instance, Intel Trust Domain Extensions (TDX) [14] creates an isolated virtual machine, called a ⑦ Trust Domain (TD), whose memory and vCPU states are cryptographically protected from the rest of the system. Running each function inside a dedicated TD provides strong confidentiality and integrity guarantees for its code and data.

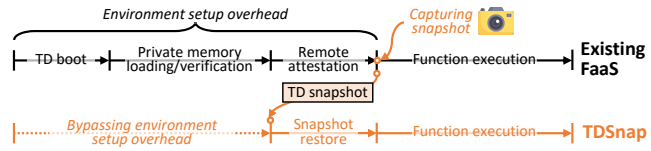


Figure 1: Cold start procedure of existing FaaS and *TDSnap*.

However, this approach introduces severe performance penalties, particularly in the function cold-start, due to the complex bootstrapping process of TDs.

As functions are executed only upon request, running each function inside a dedicated TD requires creating a function TD for every invocation. As shown in Figure 1, this process includes initializing a TD and its metadata, allocating private memory, validating code and data pages, and optionally performing ⑧ remote attestation. Repeating these steps at every function invocation results in multi-second startup delays; in our measurements, even simple functions incur ⑨ more than 4 seconds of startup time. Because practical FaaS platforms require sub-second cold start latency, this overhead renders naïve TDX integration impractical.

Previous studies [25, 29] have attempted to mitigate these costs by maintaining a shared, long-lived TD that hosts multiple functions. Although this design eliminates the overhead associated with TD initialization, it sacrifices security isolation: functions within the same shared TD are isolated by ⑩ software-level protection (e.g., namespaces), leaving them vulnerable to cross-function attacks. Thus, there remains a fundamental tension between strong isolation and fast startup in confidential FaaS environments.

This paper presents *TDSnap*, a novel TDX-compatible snapshot system that reconciles this tension by providing both hardware-backed per-function isolation and fast startup. As shown in Figure 1, *TDSnap* securely records the state of a fully initialized and verified function and its TD as a TD snapshot, which can be restored on demand. During snapshot creation, *TDSnap* generates a TD template that captures immutable TD states for rapid re-boot and identifies ⑪ working-set pages using a TD-aware page-tracking mechanism. The resulting snapshot is cryptographically protected by TDX and retained in an in-memory cache for fast restore. Upon invocation, *TDSnap* re-boots the TD by reusing the ⑫ TD template, prefetches the working-set pages into private memory, and verifies integrity and freshness through snapshot-bound attestation. This design preserves TDX's isolation guarantees for each function while substantially reducing cold-start latency.

Contributions. We make the following contributions:

- We identify key performance and security challenges in integrating snapshots with TDX to reduce function cold-start latency.
- We design and implement the first TDX-aware function snapshot system that securely captures and restores functions and their TD states, including verified working-sets.

This work is licensed under a Creative Commons Attribution 4.0 International License.
DAC '26, July 26–29, 2026, Long Beach, CA, USA
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2254-7/2026/07
https://doi.org/10.1145/3770743.3804055

학습 메모 · 원문 1쪽

① **FaaS:** 요청이 올 때만 함수를 띄워 실행하고 끝나면 지우는 클라우드 방식. 예: AWS Lambda. 서버를 계속 켜 두지 않으니 싸다.

② **CVM(기밀 VM):** CPU가 VM 메모리를 암호화해서 호스트 OS와 하이퍼바이저도 내용을 못 보게 하는 VM. Intel TDX, AMD SEV-SNP가 대표적.

③ **cold start:** 함수를 처음부터 띄우는 과정. 이 시간이 1초를 넘으면 FaaS로 쓰기 어렵다.

④ **핵심 결과:** 시작 지연 최대 40배 감소.

⑧ **원격 증명:** 장치가 자기 소프트웨어 측정값(해시)을 하드웨어 키로 서명해 외부 검증자에게 보내는 절차. → **카메라 원격 증명도 같은 개념이다.**

⑨ **문제의 크기:** 단순 함수도 TD로 띄우면 4초 이상. FaaS 목표(1초 미만)의 4배.

⑩ **기존 방식(TD 공유)의 한계:** 함수끼리 하드웨어가 아닌 namespace로만 격리된다. 한 함수가 뚫리면 같은 TD의 다른 함수도 위험.

⑤ **multi-tenant:** 서로 다른 고객(tenant)의 코드가 같은 서버에서 함께 도는 구조. 옆 고객이 공격자일 수 있다.

⑥ **위협 모델의 출발점:** 호스트 OS까지 뚫렸다고 가정한다. → **HardWhale도 같은 가정(N-comp).**

⑪ **working set(작업 집합):** 요청을 처리할 때 실제로 자주 쓰는 메모리 페이지들. 이것만 먼저 올리고 나머지는 필요할 때 올린다.

⑫ **TD 템플릿:** TD를 만들 때 매번 같은 부분(바뀌지 않는 구조)을 미리 만들어 둔 틀. 매번 새로 만들지 않아 빠르다.

⑦ **TD:** TDX가 만드는 격리 VM 하나. 메모리와 vCPU 레지스터가 하드웨어로 암호화·보호된다. → **리눅스가 못 들여다보는 보안 영역이라는 점에서 EN683의 EIP-130 안쪽과 비슷하다(비유).**

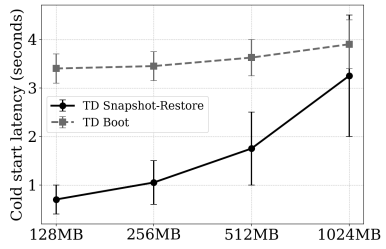


Figure 2: Cold start latency of a full TD boot and a snapshot.

- We demonstrate that *TDSnap* reduces cold-start latency for TDX by up to 40× compared to the baseline and achieves performance comparable to state-of-the-art non-TDX snapshot systems [4, 31].

2 Background and Motivation

2.1 FaaS and Intel TDX

Function-as-a-Service (FaaS). FaaS [6] is a representative paradigm of serverless computing that enables developers to compose complex applications from lightweight functions. Each function is registered to a specific event (e.g., a client request). When an event occurs, the FaaS platform creates an isolated runtime (e.g., a microVM) and loads the function binary and required libraries into it. This process, referred to as a *cold start*, denotes the setup phase in which a function is launched from scratch. The function then processes the incoming request, and upon completion, both the function instance and its allocated resources are entirely released.

Intel TDX. Intel Trust Domain Extensions (TDX) [14] is one of the most widely adopted CVM technologies in modern cloud infrastructures. It creates hardware-isolated virtual machines, called Trust Domains (TDs), protecting TDs from the rest of the system, including the host OS and VMM. A TDX module, a privileged firmware component executing in Secure Arbitration Mode (SEAM), manages the lifecycle of TDs within the server. It exposes a secure interface, known as *SEAMCALL*, through which the VMM interacts with it to perform resource scheduling and exception handling for TDs.

To create a new TD, the VMM issues a sequence of SEAMCALLs that allocate TD structures and configure its private memory. The TDX module measures and validates each page before admitting it into the private memory, ensuring that only verified code and data are loaded. All private-memory pages are protected with hardware-enforced encryption and authentication, preventing even the host OS or VMM from accessing them. Guest physical address (GPA) to host physical address (HPA) translations for this memory are managed by the Secure Extended Page Table (SEPT). Although the VMM may request SEPT updates via SEAMCALLs, every update must be explicitly validated and approved by the TD.

2.2 Integrating TDX into FaaS

In FaaS environments, multiple functions from different tenants are deployed on the same machine, sharing underlying hardware resources. This co-location introduces two major security risks. First, a malicious function can exploit shared resources (e.g., kernel exploit [19]) to access sensitive data of other functions. Second, a compromised privileged software (e.g., the host OS) can manipulate the code and data of all tenant functions by abusing its privileges. Such threats are severe for data-sensitive workloads, including facial recognition and financial processing.

Isolating each function within a dedicated TD prevents its sensitive data from leaking to or being tampered with by malicious functions or system software. However, integrating TDX into FaaS introduces significant performance challenges due to both the complexity of TDX operations and the transient nature of functions, especially during function cold-starts. When a request arrives, a new TD must be created, and the function binary along with its dependent libraries must be loaded into the TD's private memory, as shown in Figure 1. During this process, every page is measured and validated by the TDX module, substantially increasing cold-start latency. As shown by the TD-Boot plot in Figure 2, even a small function (128 MB) requires more than three seconds to launch, while a 1 GB function incurs over four seconds. Considering that typical FaaS platforms demand sub-second startup latency, such delay is unacceptable. Our analysis reveals the root cause: for each 4 KB page, the VMM must issue at least two SEAMCALLs to populate and measure the page, meaning a 1 GB function requires roughly 524K SEAMCALL invocations (100K CPU cycles per page).

Most prior studies [25, 29] mitigate the cold-start latency of CVM-backed FaaS by relying on CVM-sharing and warm-start techniques. CoFunc [29], for instance, maintains a long-lived shared TD that dynamically hosts multiple functions. Instead of creating a fresh TD per invocation, functions are loaded into this shared TD, and verified library pages already resident in the TD are reused across functions through copy-on-write (CoW) techniques.

However, such sharing-based designs have fundamental limitations that stem directly from collapsing hardware-isolated trust boundaries. Because multiple tenants execute within the same TD, isolation is enforced only through in-guest software mechanisms (e.g., namespaces) rather than TDX's hardware-based isolation. A malicious or compromised function can therefore access sensitive data of other co-located functions. Thus, a breach in one function compromises all others within that shared TD. Also, the long-lived nature of the shared TD allows attacker-controlled code to persist across invocations, enabling stealthy long-term attacks. For functions carrying sensitive data or requiring strict security policies, such trust collapse renders CVM-sharing inherently inappropriate. From a performance perspective, CVM-sharing also presents structural inefficiencies. CoW is beneficial only when tenants share identical pages. When functions require distinct libraries, CoW opportunities collapse and large amounts of private memory must be imported, incurring substantial TDX overhead. Also, if no suitably *warm* shared TD exists, a new TD should be created from scratch. Both cases lead to high startup latency (see Section 4).

2.3 Challenges in Snapshotting for TDX

In contrast to CVM-sharing approaches, this work introduces a snapshot-based mechanism for TDX that mitigates cold-start latency while preserving hardware-enforced isolation between functions. Snapshotting has been widely employed in FaaS systems [4, 8, 31] to reduce startup overhead by capturing the fully initialized state of a function into a restorable image. Applied to TDX, our system first boots a dedicated TD for a target function, loads the

학습 메모 · 원문 2쪽

① **기여 3:** TDX 기준선 대비 40배, TDX를 안 쓰는 최신 스냅샷 시스템(REAP, FaaSnap)과 비슷한 속도.

⑧ **느린 근본 원인:** 4KB 페이지마다 SEAMCALL 2번 이상. 1GB = 26만 페이지 × 2 = 52만 번, 페이지당 약 10만 CPU 사이클.

② **microVM:** 장치 에뮬레이션을 최소로 줄인 아주 가벼운 VM. 예: AWS Firecracker. 수백 ms 안에 뜬다.

⑨ **CoW:** 여러 주체가 같은 페이지를 공유하다가 누군가 쓰려 할 때만 복사본을 만드는 기법. 메모리와 시간을 아낀다.

⑩ **핵심 비판:** TD를 여러 고객이 공유하면 하드웨어가 보장하던 신뢰 경계가 무너진다.

⑪ **in-guest:** VM 안쪽 소프트웨어(게스트 커널의 namespace 등)로만 격리한다는 뜻. 게스트 커널 취약점 하나로 뚫린다.

③ **SEAM:** TDX 모듈이 도는 CPU의 특수 실행 모드. 호스트 커널보다 높은 권한으로 TD를 관리한다.

④ **SEAMCALL:** 호스트(VMM)가 TDX 모듈에 "TD 만들어 줘, 페이지 넣어 줘"라고 요청하는 CPU 명령. 시스템콜의 TDX판이라고 보면 된다.

⑤ **측정(measurement):** 올리는 페이지마다 해시를 누적 기록하고 검증. → **EN683 보안 부트에서 다음 단계 이미지의 해시·서명을 검증하는 것과 같은 원리.** 다만 여기서는 4KB 페이지 단위.

⑥ **SEPT:** TD의 게스트 주소를 실제 물리 주소로 바꾸는 페이지 테이블. 호스트가 직접 못 고치고 SEAMCALL을 거쳐야 하며, TD가 승인해야 바뀐다.

⑦ **위협 2:** 권한 높은 소프트웨어(호스트 OS)가 뚫리면 모든 고객 함수를 조작할 수 있다. TDX가 막으려는 것이 이것.

function binary and its dependencies into the TD's private memory, and verifies all loaded pages through the TDX measurement process. It then records the TD's complete state, including verified private pages, into a TD snapshot file. Upon invocations, a TD can be rapidly restored from the snapshot, bypassing the costly full TD boot and private memory initialization. Despite its clear potential, snapshots for TDX remain unexplored. We identify two challenges that must be addressed to enable function snapshots for TDX.

Lack of working-set awareness. A snapshot preserves the full memory image of a fully initialized function, which includes both cold pages used only during initialization and ① working set pages frequently accessed during request execution. To achieve low startup overhead, a snapshot system must prefetch only the working set while lazily loading cold pages; otherwise, restoration latency grows substantially [4, 8, 16, 30, 31]. However, working set identification in existing snapshot systems relies on host visibility into guest memory access behavior, an assumption that does not hold in TDX. The TDX threat model prevents the host from inspecting private pages or observing guest level memory events, which eliminates the basic signals these systems depend on. For example, ② page fault tracing is infeasible because TD page faults are handled internally without ③ VM exits, and FaaSnap's host page cache based introspection [4] cannot operate since TD private pages and SEPT metadata are inaccessible to the host. These constraints are intrinsic to TDX rather than limitations of specific mechanisms; detailed discussion is provided in Section 5. As a result, no existing approach can determine a function's working set inside a TD, forcing snapshot restoration to reload all pages and thereby inflating cold-start latency.

To show this problem, we implement a baseline TD snapshot mechanism that restores all pages from a snapshot into a TD and then measure its cold-start latency. As shown by the *TD snapshot* plot in Figure 2, a snapshot offers notable benefits for small functions. For example, with a 128 MB footprint, cold-start latency drops below one second. However, even a 256 MB function already exceeds this threshold, and the advantage diminishes rapidly as memory size increases; when the footprint reaches 1 GB, startup latency exceeds three seconds, and the ④ tail latency surpasses that of full TD boot. This overhead stems from the expensive private memory loading and verification steps of TDX, which involve multiple SEAMCALLs. Thus, reducing the number of pages reloaded during cold start is essential for making TD snapshots practical.

Lack of snapshot security mechanisms. Applying function snapshots in TDX environments introduces a security challenge that prior non TDX snapshot systems [4, 8, 31] do not consider. In FaaS, the FaaS platform is responsible for restoring snapshots during cold starts, and because the platform operates on the host side, the snapshot containing the TD state must also be stored on the host side (e.g., a file system). Under the TDX threat model, however, the host OS cannot be trusted, which exposes the stored snapshot to several risks. An attacker with host access can read snapshot contents, revealing private memory pages that may contain sensitive data such as credentials captured during snapshot creation. The attacker can also mount ⑤ rollback attacks by replacing a recent snapshot with an older version containing a known vulnerability, thus causing the TD to run an outdated binary while appearing legitimate. Since no prior work provides security mechanisms for TD snapshots, there is a clear need for a TDX compatible snapshot system that not only preserves confidentiality and integrity but also detects snapshot tampering and verifies that the restored state corresponds to the originally attested TD.

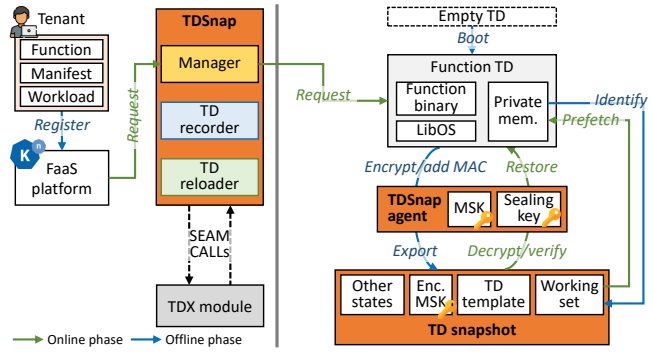


Figure 3: The overall architecture of *TDSnap*.

3 *TDSnap* Design

3.1 Threat Model

Similar to previous studies in CVMs [12, 17], ⑥ we trust the TDX module and Intel-provisioned ⑦ service TDs [1] that the platform uses to support TDX functionality. We assume that an attacker can (1) compromise the privileged software (e.g., the host OS) and (2) deploy a malicious function. The attacker's goal is to inspect or modify the private memory state or vCPU registers of other functions. The security goal of *TDSnap* is to ensure that each function's TD maintains confidentiality and integrity of its private memory against a compromised host or other functions. We exclude attacks that exploit known vulnerabilities in a function binary itself, as TDX does not aim to mitigate bugs within the guest software [7].

3.2 System Architecture

TDSnap implements a TDX-compliant snapshot mechanism that relies solely on SEAMCALLs. Specifically, it repurposes the TD ⑧ migration feature [2], originally designed for live TD migration, to support secure snapshot creation and restoration. As shown in Figure 3, the workflow consists of an offline phase (blue arrows) and an ⑨ online phase (green arrows). In the offline phase, *TDSnap* exports a function TD's execution state into a snapshot file and performs TD-aware page tracking to construct working-set pages. The generated snapshot is then ⑩ encrypted and authenticated before being stored on the host file system. In the online phase, *TDSnap* restores the snapshot, prefetches the working-set pages, and performs snapshot-bound attestation to verify the integrity and freshness of the reconstructed TD. The function can handle requests immediately, while missing pages are fetched on demand by *TDSnap*. This design significantly reduces cold-start latency while preserving per-function isolation and full compatibility with TDX.

3.3 Offline Phase: TD Snapshot Creation

In the offline phase, a tenant registers a function by providing its binary, manifest, and a sample workload. The TD recorder is responsible for generating a TD snapshot for each registered function. As shown in Figure 3, it boots a fresh TD with our TDX-compatible LibOS [17] and loads the function binary and libraries according to

학습 메모 · 원문 3쪽

① **도전 1:** 기존 스냅샷 시스템은 호스트가 게스트의 메모리 접근을 엿봐서 working set을 찾는다. TDX에서는 그게 금지된다.

② **page fault:** 아직 메모리에 없는 페이지에 접근할 때 나는 예외. 보통 이걸 추적해서 어떤 페이지를 쓰는지 안다. TD 안의 fault는 호스트에 안 보인다.

⑥ **신뢰 대상(TCB):** TDX 모듈(CPU 안에서 SEAM 모드로 도는 인텔 서명 펌웨어)과 인텔 service TD만 믿는다. 호스트 OS는 믿지 않는다. → **EN683에서 BootROM과 OTP만 믿는 것과 같은 구도.**

③ **VM exit:** VM 실행이 멈추고 제어가 호스트(VMM)로 넘어오는 사건. 호스트가 무언가를 관찰할 수 있는 유일한 순간이다.

⑦ **service TD:** 인텔이 제공하는 특수 TD. 마이그레이션 같은 플랫폼 기능을 대신 수행한다. TDSnap 에이전트가 이것으로 만들어진다.

⑧ **핵심 아이디어:** 원래 TD를 다른 서버로 옮기려고 만든 마이그레이션 기능(EXPORT/IMPORT SEAMCALL)을 스냅샷 저장·복원에 재활용한다. 새 하드웨어 없이 표준 TDX만으로 동작.

④ **tail latency:** 지연 분포의 꼬리(P99 등). 평균이 좋아도 가끔 아주 느리면 서비스 품질이 나쁘다.

⑨ **offline / online phase:** 미리 스냅샷을 만드는 단계(offline)와 요청이 왔을 때 복원하는 단계(online). 비싼 일은 offline으로 미룬다.

⑩ **스냅샷 보호:** 암호화(내용을 못 봄) + 인증 MAC(바꾸면 들킴). 호스트 디스크에 저장해도 안전하다.

⑤ **롤백 공격:** 최신 스냅샷을 알려진 취약점이 있는 옛 버전으로 바꿔치기. → **펌웨어 다운그레이드 공격과 같다.** 서명은 옛 버전도 유효하므로 버전 카운터가 따로 필요하다.

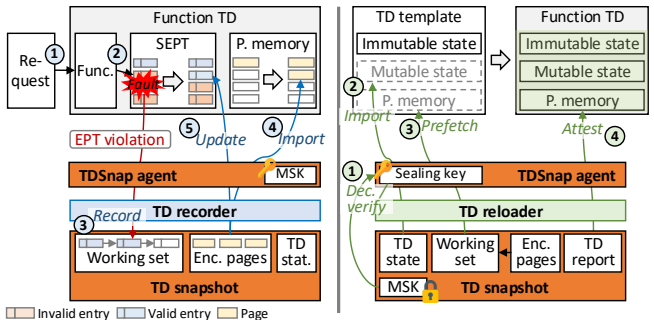


Figure 4: The (S)EPT violation tracking mechanism (left) and snapshot-based function TD restoration (right).

the manifest, which specifies the function's required runtime configuration (e.g., memory layout), thereby populating the TD's private memory and SEPT with code and data pages. The TD recorder may optionally perform remote attestation to confirm the integrity of this initialized environment. Next, the Manager creates a verified *TDSnap* agent that operates as a service TD [1] to handle TD migration interfaces. Through the TDX module, this agent obtains controlled access to the function TD's private memory and derives a Migration Session Key (MSK). The TD recorder then suspends the function TD and exports its private pages via EXPORT SEAMCALLs [2]. As shown in Figure 3, the agent encrypts each page and computes a MAC using the MSK before writing them to the host file system, producing an initial TD snapshot that securely captures both the TD's immutable state and its private memory.

Working-set identification. Unlike previous snapshot systems, *TDSnap* introduces a working-set identification mechanism that operates fully within TDX constraints by exploiting architectural properties unique to TD migration. Under normal execution, page faults inside a function TD do not trigger TD exits and are resolved internally, leaving the VMM unable to observe memory access patterns. However, during TD migration, an EXPORT SEAMCALL flushes all SEPT entries, and the first access to any private page subsequently triggers an (S)EPT violation. Unlike ordinary page faults, these violations necessarily cause VM exits, allowing the VMM to observe the faulting GPA while still preventing access to the page contents. This behavior provides a unique and TDX-compliant visibility channel that forms the basis of our design. Figure 4 illustrates the process. ① The TD recorder initiates tenant-provided workloads, allowing the function TD to process a request. ② On each first access after a SEPT flush, the TD raises an (S)EPT violation, which results in a VM exit. ③ The recorder logs the GPA from the exit context, retrieves the corresponding page from the initial snapshot, and invokes an IMPORT SEAMCALL. ④ The migration agent decrypts and imports the page, and ⑤ updates the SEPT entry accordingly. After completing the request, the TD recorder flushes SEPT entries and repeats the process, aggregating pages consistently accessed across iterations into the working-set.

Snapshot sealing. After profiling the working-set, the TD recorder exports the remaining TD mutable state (e.g., vCPU registers) into the TD snapshot. Because all snapshot contents are encrypted with the MSK, this key must also be stored alongside the snapshot for future restoration. However, the MSK is intended solely for TD migration and is not designed to be persistently stored in the untrusted host file system. To securely store the MSK, *TDSnap* leverages TDX's built-in key-management mechanism. The agent obtains a sealing key from the TDX module; this key is cryptographically bound to the agent's identity and is never exposed outside the processor. The agent encrypts the MSK with the sealing key and embeds the sealed MSK in the snapshot header. As only the same attested agent can later derive the sealing key, we ensure that no other entity, including the host OS, can decrypt or use the snapshot. As the output of the offline phase, *TDSnap* produces a TD snapshot that captures a function TD's complete execution state. The snapshot consists of the encrypted MSK, TD mutable state, the working-set, private-memory pages, and the attestation report. Among them, the working-set is retained in a per-function in-memory cache to support fast prefetching. The remaining immutable state (e.g., the SEPT root and TDVPS), which does not change after TD creation, is also preserved as a TD template in memory for fast TD re-boot.

3.4 Online Phase: TD Snapshot Restore

In the online phase, the FaaS platform handles function invocations by restoring TD snapshots. Upon receiving a request, the platform forwards it to the Manager, which temporarily holds the request and initiates the restoration executed by the TD reloader. It first locates the corresponding snapshot and instructs the agent. As shown in the right part of Figure 4, the agent obtains the sealing key and then ① decrypts the snapshot's MSK to validate both the integrity digest and the freshness tag derived from a hardware monotonic counter, preventing rollback attacks and ensuring that the snapshot has not been modified. The reloader then retrieves the corresponding TD template and ② populates it with the TD mutable state extracted from the snapshot, avoiding the overhead of re-creating immutable TD states. It subsequently ③ prefetches the working-set pages into private memory using IMPORT SEAMCALLs. For each page, the agent decrypts the content and verifies its MAC via the MSK before importing it. Because the working-set is maintained in an in-memory cache, *TDSnap* avoids disk I/O during prefetching. Note that pages decrypted with the MSK are never stored in plaintext inside the TD. When written into private memory, the hardware engine re-encrypts them with the TD's private key.

Snapshot-bounded attestation. Once the working-set is restored, *TDSnap* performs snapshot-bounded attestation to ensure that the reconstructed TD matches the attested state captured during snapshot generation. The agent retrieves the verified hash-based tag of the snapshot, which reflects the TD's current state (e.g., TDCS and MRTD/RTMR values). As shown by ④, it acts as a proxy attester and verifies that the TDREPORT matches the TD identity and configuration recorded at snapshot creation. Because the snapshot remains immutable, and all pages are MAC-protected under the MSK, the agent also authenticates each imported page. This mechanism ensures that without re-executing the full boot sequence, the attestation report reflects a previously verified software stack, maintaining trust guarantees for the function owner.

On-demand paging. After these steps, the TD template becomes a fully restored and verified function TD, and the Manager forwards the pending request to it. During execution, any access to a non-prefetched page triggers a controlled on-demand fault, which the TD reloader resolves by securely importing the missing page.

학습 메모 · 원문 4쪽

⑧ **sealing key(봉인 키):** CPU 안에서 특정 소프트웨어 신원에 묶여 생성되는 키. 같은 에이전트만 다시 얻을 수 있다. → **EN683의 HUK(하드웨어 고유키)로 키를 감싸 저장하는 방식과 같은 발상(추정).**

⑨ MSK를 sealing key로 암호화해서 스냅샷 헤더에 넣는다. 호스트 디스크에 있어도 호스트는 MSK를 꺼낼 수 없다.

① **(S)EPT violation:** 페이지 테이블에 유효한 항목이 없을 때 나는 예외. 일반 page fault와 달리 반드시 VM exit를 일으켜 호스트가 주소(GPA)를 볼 수 있다. 내용은 여전히 못 본다.

② **MSK(마이그레이션 세션 키):** TD를 옮길 때 페이지를 암호화하려고 TDX 모듈과 에이전트가 만드는 대칭키. TDSnap은 이 키로 스냅샷 페이지를 암호화하고 MAC을 붙인다.

③ **EXPORT SEAMCALL:** TD의 private 페이지를 암호화된 형태로 TD 밖으로 내보내는 마이그레이션 명령. 짝은 IMPORT(다시 넣기).

④ **MAC:** 키를 가진 쪽만 만들 수 있는 짧은 검증값. 내용이 한 비트라도 바뀌면 검증이 실패해서 변조를 잡는다.

⑩ **monotonic counter:** 증가만 하고 되돌릴 수 없는 하드웨어 카운터. 스냅샷에 이 값을 신선도 태그로 묶어서 옛 스냅샷 재사용(롤백)을 막는다. → **펌웨어 anti-rollback 버전 카운터(OTP)와 같은 역할.**

⑤ **핵심 관찰:** EXPORT를 하면 SEPT 항목이 전부 비워진다. 그래서 다음 첫 접근마다 예외가 난다.

⑥ **GPA(Guest Physical Address):** 게스트가 생각하는 물리 주소. 호스트는 어느 페이지를 건드렸는지(주소)만 알고 데이터는 모른다.

⑪ **MRTD:** TD 생성 시 올린 초기 내용의 측정값. RTMR: 실행 중 추가 측정을 누적하는 레지스터(TPM의 PCR과 비슷). 둘이 TD의 신원이 된다.

⑫ **snapshot-bounded attestation:** 부팅을 다시 하지 않아도, 복원된 TD의 TDREPORT가 스냅샷 생성 당시 증명된 신원과 같은지 에이전트가 대신 확인한다. → **카메라 원격 증명에도 비슷한 구조를 생각해 볼 수 있다(추정).**

⑦ **working set 결정:** 요청을 여러 번 돌리며 매번 SEPT를 비우고, 반복해서 접근된 페이지만 모은다.

⑬ **on-demand paging:** working set에 없는 페이지는 접근할 때 그때그때 IMPORT로 가져온다.

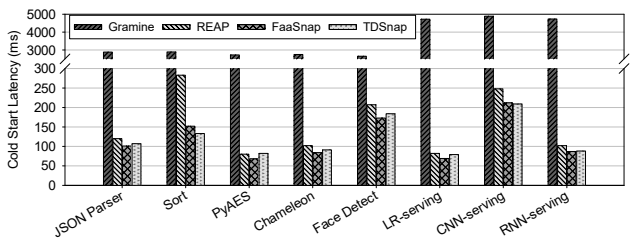


Figure 5: Cold-start latency of each function.

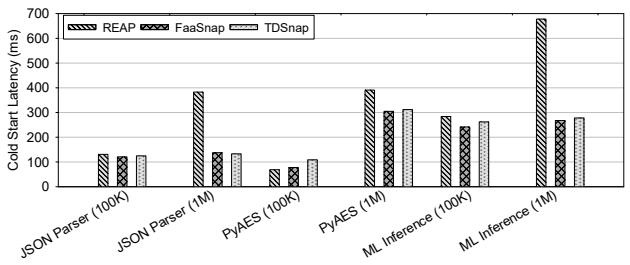


Figure 6: Cold-start latency variation on different input sizes.

Once the request completes, the Manager flushes the function TD's mutable state and converts it back into a TD template.

4 Evaluation

4.1 Evaluation environment

We implement a full prototype integrated with ① Knative and extend ② QEMU to support TDX migration. *TDSnap*'s Manager, TDRecorder, and TDLoader are implemented as VMM-level plugins, while the *TDSnap* agent is built using Intel's ③ Migration TD [1]. All experiments are conducted in a real-world FaaS environment using two TDX-enabled servers (④ Xeon 6544Y with TDX 1.5). Guest TDs are provisioned with 4 vCPUs and 1 GB memory, matching typical FaaS settings. For workloads, we select eight functions from FunctionBench [15], each packaged as a ⑤ Gramine-TDX manifest. We compare *TDSnap* against two baselines: vanilla Gramine-TDX [17] as a lower-bound baseline without TD snapshots, and state-of-the-art non-CVM snapshot systems (REAP [31] and FaaSnap [4]) as upper-bound baselines without the overhead of TDX.

4.2 End-to-end performance

We first measure the cold-start latency of each selected function. As shown in Figure 5, *TDSnap* achieves ⑥ 121 ms for simple functions (e.g., JSON parser) and 209 ms for complex functions (e.g., RNN serving) on average, showing an order-of-magnitude lower latency than Gramine-TDX. This improvement stems from our optimized snapshot-restore, which combines TD templates with working-set prefetching. Compared to ⑦ REAP and FaaSnap, *TDSnap* delivers comparable performance despite the overheads imposed by TDX. REAP suffers from disk I/O overheads during working-set prefetching, whereas *TDSnap* avoids such overheads by storing compact working sets in an in-memory cache. FaaSnap mitigates disk I/O through concurrent loading, but this optimization is incompatible with TDX because private-memory imports and SEPT updates must follow ⑧ strict sequential SEAMCALL operations. *TDSnap* preserves similar performance by maintaining working-set pages in an in-memory cache at low memory cost (see Section 4.3). We also evaluate working-set stability under varying input sizes. Snapshots generated with 100 KB inputs are evaluated with both 100 KB and 1 MB requests. As shown in Figure 6, *TDSnap* maintains performance comparable to non-CVM snapshot systems, showing that *TDSnap* identifies stable working sets across function TD invocations.

Table 1: TD booting latency comparison.

Step	From scratch	TD template
TD creation	355ms	-
SEPT init	263ms	-
Metadata setup	98ms	-
Working set restore	140ms	140ms
Total	856ms	140ms

Comparison with CVM-sharing. We attempted to compare *TDSnap* with ⑨ CoFunc; however, its open-source artifact does not include a TDX implementation. We therefore rely on the published evaluation results (Section 7.2) from the CoFunc paper [29]. When executing the Face Detect function, CoFunc reports 17 ms latency by reusing a long-lived shared TD and forking required library pages from co-located functions. These optimizations, however, depend on the presence of an already active shared TD populated with reusable library pages. When such a TD is unavailable, either because no functions are running or usable libraries do not overlap, startup latency sharply increases to 657 ms due to the substantial overhead of private-memory loading under TDX. In contrast, *TDSnap* consistently achieves ⑩ 184 ms latency (3× faster) by restoring the Face Detect function from its TD snapshot, without requiring a persistent shared TD or assuming inter-function library sharing.

4.3 Microbenchmark

TD template. *TDSnap* employs TD templates to eliminate the cost of reconstructing immutable TD state during each invocation. As shown in Table 1, full TD booting incurs substantial overhead from creating the SEPT root and TDCS metadata. In contrast, a TD template preserves these immutable states and restores only mutable states from the template, reducing TD boot latency by ⑪ 6×. Although maintaining TD immutable state introduces persistent memory usage, the cost is negligible: for a 4-vCPU TD with 256 MB of memory, the TD template occupies only ≈0.14 MB per function.

Working-set size. We next evaluate the impact of our EPT-violation tracking, which enables *TDSnap* to prefetch only the working-set pages rather than restoring the full-memory initial snapshot. Table 2 reports the ratio between the initial snapshot and the extracted working set. For simple functions, *TDSnap* reduces the memory footprint to ⑫ 3.3–7.5%, indicating that most pages accessed during initialization are unnecessary for request processing and are correctly excluded as cold pages. Even for memory-intensive functions, the working set remains compact at 12–15%, amounting to only a few tens of megabytes (about 30 MB). Maintaining these working sets in an in-memory cache enables fast prefetching with negligible persistent memory overhead. Moreover, *TDSnap* maintains one working-set cache per TD snapshot rather than per function instance, ensuring that memory usage remains stable regardless of the number of concurrently executing function TDs.

Working-set accuracy. The right-most columns of Table 2 present the consistency ratio and the number of on-demand page faults,

학습 메모 · 원문 5쪽

⑨ **CoFunc:** TD를 공유하는 기존 연구(USENIX Security 2025). TDX 구현이 공개되지 않아 논문 수치로 비교했다.

⑩ **Face Detect:** CoFunc 공유 TD가 없을 때 657 ms vs TDSnap 184 ms(3배). 공유 TD가 있으면 CoFunc는 17 ms지만 보안 격리를 희생한다.

① **Knative:** 쿠버네티스 위에서 FaaS를 돌리는 오픈소스 플랫폼.

② **QEMU:** 리눅스에서 VM을 실행하는 가상화 소프트웨어(VMM 역할). TDX 마이그레이션을 지원하도록 확장했다.

③ **Migration TD (MigTD):** 인텔이 공개한 Rust 기반 service TD. TDSnap 에이전트가 이걸 바탕으로 만들어졌다.

④ **실험 환경:** TDX 1.5 서버 2대, TD당 4 vCPU·1GB. 마이그레이션 기능은 TDX 1.5부터라 이전 세대 CPU에서는 못 쓴다(추정).

⑤ **Gramine-TDX:** TD 안에서 앱 하나를 돌리는 가벼운 라이브러리 OS. 스냅샷 없는 하한 기준선.

⑪ **표 1:** TD 템플릿으로 부팅 856 ms → **140 ms(6배).** 템플릿 메모리는 함수당 약 0.14 MB.

⑥ **결과:** 단순 함수 121 ms, 복잡한 함수(RNN) 209 ms. Gramine-TDX보다 한 자릿수 이상 빠르다.

⑫ **working set 크기:** 단순 함수는 전체의 3.3~7.5%, 복잡한 함수도 12~15%(약 30 MB). 그래서 메모리 캐시에 둘 수 있다.

⑦ **REAP, FaaSnap:** TDX를 쓰지 않는 최신 스냅샷 시스템. 보안이 없는 대신 빠른 상한 기준선.

⑧ **TDX 제약:** private 메모리 IMPORT와 SEPT 갱신은 순서대로만 해야 해서 FaaSnap식 병렬 로딩을 못 쓴다.

Table 2: Working-set ratio (size), consistency, and page-fault statistics. Values in the parentheses are standard deviation.

Category	Function	Working-set ratio	Consist. ratio	Page faults
Simple	JSON parser	3.3% (8.4MB)	98.5%	48 (5)
	Sort	3.6% (9.2MB)	97.2%	69 (7)
	PyAES	4.8% (12.2MB)	96.8%	109 (10)
	Chameleon	4.4% (11.2MB)	97.0%	84 (8)
Complex	Face detect	7.5% (19.2MB)	92.1%	369 (33)
	LR serving	14.4% (36.8MB)	94.6%	244 (38)
	CNN serving	12.9% (33.0MB)	89.5%	402 (24)
	RNN serving	14.5% (37.1MB)	93.8%	328 (49)

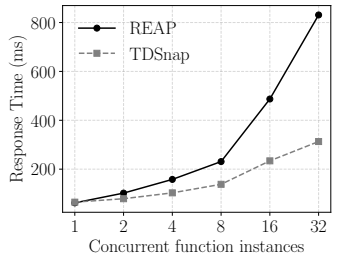


Figure 7: Concurrent function startup latency.

both of which quantify working-set accuracy. On average, *TDSnap* achieves a ① 97.3% consistency ratio for simple functions and 92.5% for complex functions, demonstrating that EPT-violation tracking reliably identifies pages reused across multiple invocations. To further assess accuracy, we measure on-demand page faults that occur when a function accesses pages not included in its working-set. *TDSnap* incurs fewer than 50 such faults for simple workloads and fewer than 410 for complex ones. Across all benchmarks, more than 95% of accessed pages are included in the working-set. This confirms that *TDSnap* captures the dominant access patterns with high precision, minimizing on-demand faults during execution.

4.4 Concurrent Function Startup

Finally, we evaluate scalability under concurrent cold-starts. Figure 7 shows cold-start latency for parallel initialization of HelloWorld functions. REAP exhibits ② exponential slowdowns, from 62 ms (1 instance) to 831 ms (32 instances), due to increased disk I/O. In contrast, *TDSnap* scales linearly, increasing only from 65 ms to 313 ms across the same concurrency range. This improvement stems from the use of TD templates, which reduce SEAMCALL invocations during TD booting, and in-memory working sets, which eliminate disk I/Os during snapshot restoration. These results show that *TDSnap* preserves low cold-start latency even under bursty workloads while maintaining TDX's security guarantees.

5 Related Work

Function snapshots. A variety of snapshot systems [4, 8, 30, 31] have been proposed to reduce cold-start latency in FaaS environments. REAP [31] employs a customized page-fault handler to record working-set pages and persist them to the local file system, while FaaSnap [4] refines this approach through locality-aware working-set detection and concurrent prefetching. However, none of them are designed with TDX's threat model in mind. As a result, their working-set identification methods cannot be applied to function TDs. For example, ③ custom fault handlers inside a TD cannot safely export monitored GPAs without introducing custom I/O channels that enlarge the attack surface, nor can these systems protect the resulting snapshots from the untrusted host OS or VMM. In contrast, *TDSnap* is the first system to leverage TDX migration SEAMCALLs to implement a TDX-compliant snapshot mechanism.

CVMs for FaaS. Several recent systems [25, 29] apply TDX or other CVMs to FaaS using CVM-sharing and warm-start techniques, but these designs exhibit fundamental security and performance limitations (see Section 2.2 and Section 4). ④ SEVeriFast [12] improves SEV MicroVM boot latency via a specialized minimal bootstrap path, yet it focuses solely on CVM boot-time optimizations and does not support fast function-initialization mechanisms such as snapshot-based restarts and working-set extraction. Earlier ⑤ SGX-based systems such as PIE [18], ⑥ Cryonics [16], and reusable-enclave designs [32] accelerate enclave startup through snapshot or rewind techniques, but their approaches rely on SGX's process-level semantics and therefore do not generalize to TDX's VM-level architecture. In contrast, *TDSnap* is the first system to design a snapshot mechanism that operates entirely within TDX's boundary.

6 Discussions and Limitations

Target functions. *TDSnap* enables fast snapshot-based TD startup with minimal in-memory caching, allowing snapshots to be created for all registered functions if desired. Nevertheless, combining *TDSnap* with warm-start techniques can further optimize resource usage. Production traces [28] show that ⑦ fewer than 10% of functions are invoked every minute; these highly active functions can justify remaining warm in memory despite the associated memory cost. For the remaining majority of functions, which exhibit normal invocation frequencies, *TDSnap* offers an effective alternative by storing them as TD snapshots and restoring them on demand. This avoids the consistent memory cost of warm instances while still delivering low-latency startup. Finally, very cold functions (about ⑧ 3.5%), invoked less than once per week, can have their working-set evicted from the in-memory cache and placed on disk.

Function chaining. A promising direction for future work is extending *TDSnap* to support fast inter-TD communication. Although *TDSnap* accelerates individual function startup, FaaS workloads may require low-latency communication among functions to support function chaining. Integrating *TDSnap* with emerging ⑨ TDX Connect [3] enables secure, high-performance communication channels between function TDs whose state can also be captured within their TD snapshots, eliminating repeated setup overheads and further improving end-to-end performance.

7 Conclusion

This work presents *TDSnap*, the first TDX-compatible snapshot system that enables secure and fast initialization of serverless functions. By leveraging TD migration SEAMCALLs, *TDSnap* supports protected snapshot generation and reliable restoration with accurate working-set identification. Evaluations show that *TDSnap* reduces cold-start latency by up to 40× compared to the baseline.

8 ACKNOWLEDGEMENT

We thank the anonymous reviewers. This work was supported by the National Research Foundation of Korea (NRF) grant funded by the Korea government(MSIT) (No. RS-2026-25496281).

학습 메모 · 원문 6쪽

④ **SEVeriFast:** AMD SEV microVM 부팅 경로를 줄인 연구. 부팅만 빠르게 하고 스냅샷은 다루지 않는다.

⑤ **SGX:** 인텔의 이전 TEE. 프로세스 일부(enclave)만 보호한다. TDX는 VM 전체를 보호한다는 점이 다르다.

⑥ **Cryonics (SoCC 2023):** 같은 저자들(김성중, 유명성 교수님, 신승원 교수님)의 SGX enclave 스냅샷 연구. TDSnap은 이를 VM 단위 TDX로 옮긴 후속작으로 읽힌다(추정).

① **정확도:** 일관성 비율(여러 번 호출에서 같은 페이지가 쓰인 비율) 단순 97.3%, 복잡 92.5%. 접근 페이지의 95% 이상이 working set에 포함.

⑦ **한계 1:** 매분 호출되는 함수(10% 미만)는 warm 상태 유지가 더 낫다. TDSnap은 나머지 대다수 함수에 맞는 방법이다.

⑧ 아주 드물게(주 1회 미만) 호출되는 함수 약 3.5%는 working set을 디스크로 내려도 된다.

② 동시 32개 실행 시 REAP는 62 → **831 ms로 급증(디스크 I/O 경쟁).** TDSnap은 65 → **313 ms로 선형 증가.**

⑨ **TDX Connect:** TD와 장치(GPU, NIC 등) 사이를 신뢰 채널로 잇는 인텔의 차기 기능. 함수 체이닝 통신에 쓰는 것이 향후 과제.

③ **기존 방식을 TD에 못 쓰는 이유:** TD 안에 감시 코드를 넣으면 주소를 밖으로 내보낼 별도 통로가 필요하고, 그만큼 공격면이 넓어진다.

References

[1] A rust version of Migration TD. https://github.com/intel/MigTD.

[2] Intel TDX Module Architecture Specification: TD Migration. https://cdrdv2-public.intel.com/733578/intel-tdx-module-1.5-td-migration-spec-348550002.pdf.

[3] Intel® TDX connect architecture specification. https://cdrdv2-public.intel.com/773614/intel-tdx-connect-architecture-specification.pdf.

[4] Lixiang Ao, George Porter, and Geoffrey M. Voelker. FaaSnap: FaaS made fast using snapshot-based VMs. In *EuroSys*, 2022.

[5] AWS. Aws lambda. https://aws.amazon.com/lambda.

[6] Paul Castro, Vatche Ishakian, Vinod Muthusamy, and Aleksander Slominski. The rise of serverless computing. *Communications of the ACM*, 62(12):44–54, 2019.

[7] Pau-Chen Cheng, Wojciech Ozga, Enriquillo Valdez, Salman Ahmed, Zhongshu Gu, Hani Jamjoom, Hubertus Franke, and James Bottomley. Intel tdx demystified: A top-down approach. *ACM Computing Surveys*, 56(9):1–33, 2024.

[8] Dong Du, Tianyi Yu, Yubin Xia, Binyu Zang, Guanglu Yan, Chenggang Qin, Qixuan Wu, and Haibo Chen. Catalyzer: Sub-millisecond startup for serverless computing with initialization-less booting. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*, pages 467–481, 2020.

[9] Muhammed Golec, Ridvan Ozturac, Zahra Pooranian, Sukhpal Singh Gill, and Rajkumar Buyya. Ifaasbus: A security-and privacy-based lightweight framework for serverless computing using iot and machine learning. *IEEE Transactions on Industrial Informatics*, 18(5):3522–3529, 2021.

[10] Google. Collection of sample apps showcasing popular use cases using Cloud Functions for Firebase. https://github.com/firebase/functions-samples.

[11] Google. Google cloud functions.

[12] Benjamin Holmes, Jason Waterman, and Dan Williams. Severifast: Minimizing the root of trust for fast startup of sev microvms. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, pages 1045–1060, 2024.

[13] IBM. Serverless computing in financial services. https://www.ibm.com/downloads/cas/Q93RMQMN.

[14] Intel. Intel® trust domain extensions (intel tdx).

[15] Jeongchul Kim and Kyungyong Lee. Functionbench: A suite of workloads for serverless cloud function service. In *Proceedings of IEEE International Conference on Cloud Computing (CLOUD '19)*, 2019.

[16] Seong-Joong Kim, Myoungsung You, Byoungjun Kim, and Seungwon Shin. Cryonics: Trustworthy Function-as-a-Service using Snapshot-based Enclaves. In *Proceedings of the ACM Symposium on Cloud Computing (SoCC)*, 2023.

[17] Dmitrii Kuvaiskii, Dimitrios Stavrakakis, Kailun Qin, Cedric Xing, Pramod Bhatotia, and Mona Vij. Gramine-TDX: A Lightweight OS Kernel for Confidential VMs. In *ACM CCS*, 2024.

[18] Mingyu Li, Yubin Xia, and Haibo Chen. Confidential serverless made efficient with plug-in enclaves. In *Proceedings of the International Symposium on Computer Architecture (ISCA)*, 2021.

[19] Xing Li, Xue Leng, and Yan Chen. Securing serverless computing: Challenges, solutions, and opportunities. *IEEE Network*, 37(2):166–173, 2022.

[20] Oliviu Matei, Rudolf Erdei, Alexandru Moga, and Robert Heb. A serverless architecture for a wearable face recognition application. In *International Conference on Pattern Recognition*, pages 642–655. Springer, 2021.

[21] Microsoft. Azure functions.

[22] Arttu Paju, Muhammad Owais Javed, Juha Nurmi, Juha Savimäki, Brian McGillion, and Billy Bob Brumley. Sok: A systematic review of tee usage for developing trusted applications. In *Proceedings of the 18th International Conference on Availability, Reliability and Security*, pages 1–15, 2023.

[23] Joongun Park, Seunghyo Kang, Sanghyeon Lee, Taehoon Kim, Jongse Park, Youngjin Kwon, and Jaehyuk Huh. Hardware-hardened sandbox enclaves for trusted serverless computing. *ACM Transactions on Architecture and Code Optimization*, 21(1):1–25, 2024.

[24] Maryam Rostamipoor, Seyedhamed Ghavamnia, and Michalis Polychronakis. Leakless: Selective data protection against memory leakage attacks for serverless platforms. In *Proceedings of the Network and Distributed System Security Symposium (NDSS), San Diego, CA*, 2025.

[25] Patrick Sabanic, Masanori Misono, Teofil Bodea, Julian Pritzi, Michael Hackl, Dimitrios Stavrakakis, and Pramod Bhatotia. Confidential serverless computing. *arXiv preprint arXiv:2504.21518*, 2025.

[26] Martin Schwarzl, Pietro Borrello, Andreas Kogler, Kenton Varda, Thomas Schuster, Michael Schwarz, and Daniel Gruss. Robust and scalable process isolation against spectre in the cloud. In *European Symposium on Research in Computer Security*, pages 167–186. Springer, 2022.

[27] Mohammad Shahrad, Jonathan Balkind, and David Wentzlaff. Architectural implications of function-as-a-service computing. In *Proceedings of the 52nd annual IEEE/ACM international symposium on microarchitecture*, pages 1063–1075, 2019.

[28] Mohammad Shahrad, Rodrigo Fonseca, Inigo Goiri, Gohar Chaudhry, Paul Batum, Jason Cooke, Eduardo Laureano, Colby Tresness, Mark Russinovich, and Ricardo Bianchini. Serverless in the wild: Characterizing and optimizing the serverless workload at a large cloud provider. In *2020 USENIX annual technical conference (USENIX ATC 20)*, pages 205–218, 2020.

[29] Jiacheng Shi, Jinyu Gu, Yubin Xia, and Haibo Chen. Serverless Functions Made Confidential and Efficient with Split Containers. In *USENIX Security*, 2025.

[30] Simon Shillaker and Peter Pietzuch. Faasm: Lightweight isolation for efficient stateful serverless computing. In *2020 USENIX Annual Technical Conference (USENIX ATC 20)*, pages 419–433, 2020.

[31] Dmitrii Ustiugov, Plamen Petrov, Marios Kogias, Edouard Bugnion, and Boris Grot. Benchmarking, Analysis, and Optimization of Serverless Function Snapshots. In *Proceedings of the ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2021.

[32] Shixuan Zhao, Pinshen Xu, Guoxing Chen, Mengya Zhang, Yinqian Zhang, and Zhiqiang Lin. Reusable enclaves for confidential serverless computing. In *32nd USENIX security symposium (USENIX Security 23)*, pages 4015–4032, 2023.

학습 메모 · 원문 7쪽

① **참고 [1]:** 인텔 MigTD 소스. 실제 구현을 따라가 보려면 여기서 시작한다.

② **참고 [16]:** 유명성 교수님 공저 Cryonics. 같은 연구 흐름의 전작이다.